

Overcoming catastrophic forgetting in robotic manipulation via knowledge-compositional reinforcement learning

Wenzhang Liu^{a, b}, Wanyi Yao^{a, b}, Keke Yang^{a, b}, Lu Ren^{a, b}, Changyin Sun^{a, b, *} 

^a School of Artificial Intelligence, Anhui University, Hefei, Anhui, 230601, China

^b Engineering Research Center of Autonomous Unmanned System Technology, Ministry of Education, Anhui University, Hefei, Anhui, 230601, China

ARTICLE INFO

Communicated by D. Ding

Keywords:

Deep reinforcement learning
Knowledge composition
Robotic manipulation
Sequential multi-task
Catastrophic forgetting

ABSTRACT

Deep reinforcement learning (DRL) for continuous robotic manipulation faces a critical challenge in catastrophic forgetting, where models gradually lose previously learned skills when acquiring new tasks. While traditional multi-model control methods address this issue, they encounter other limitations, such as gradient conflicts in continuous learning scenarios. In this paper, we propose knowledge-compositional reinforcement learning (KCRL), a framework that trains sequential robotic manipulation tasks continuously via parameter composition. KCRL explicitly separates the parameters into task-agnostic and task-specific components, allowing the agent to extract useful knowledge for learning new tasks. Hence, it mitigates catastrophic forgetting and reduces gradient conflicts common in conventional fully shared networks. Specifically, we formulate robotic manipulation as a temporally dependent sequential decision-making problem, and extend the soft actor-critic (SAC) algorithm following the KCRL framework. Our experimental results show that the method achieves robust and adaptive manipulation in dynamic environments, significantly enhancing learning stability and efficiency across sequential tasks. The results highlight the capability of KCRL to preserve knowledge across sequential manipulation tasks and overcome catastrophic forgetting in continual robotic learning.

1. Introduction

Robotic manipulation has seen remarkable advancements in recent years, particularly in dynamic and complex environments. These developments have significantly expanded the applicability of robotic systems in fields such as industrial automation, logistics, and healthcare [1,2]. However, existing approaches still face challenges in achieving real-time adaptability and high success rates in dynamic settings. Many existing frameworks rely on multi-model control architectures that decompose the task into subtasks such as object detection, pose estimation, motion planning, and control [3–6]. These approaches mitigate interference by training a separate model for each task. However, they suffer from poor knowledge transfer, high memory or computational cost, and lack of scalability for sequential training in robotic manipulation systems. More importantly, these methods do not explicitly address gradient conflicts during sequential training, since parameter updates remain isolated per model rather than coordinated [7–9].

Robotic manipulation tasks, such as pick-and-place, are inherently sequential decision-making processes, where each subtask (e.g.,

reaching, grasping, and placing) depends on the successful completion of the previous one. A typical pick-and-place workflow involves three critical phases: (1) precise positioning of the end-effector near the target object, (2) stable grasping and object retention, and (3) accurate placement at the desired location. The complexity of these tasks is mainly due to two key challenges: i) the need to develop a time-sensitive model that ensures millisecond-level precision in dynamic environments, and ii) the requirement to generate real-time and multi-objective optimization trajectories that satisfy both kinematic constraints and task-specific success criteria.

To address these issues, deep reinforcement learning (DRL) has emerged as a promising alternative, enabling end-to-end learning of manipulation policies without the need for explicit physical modeling [10–12]. By interacting continuously with the environment, DRL-based agents can autonomously learn optimal policies that efficiently complete tasks. However, when applied to sequential manipulation tasks, existing DRL methods encounter substantial challenges. One of the most critical limitations is catastrophic forgetting, in which previously learned knowledge is overwritten during training on new tasks, causing

* Corresponding author at: School of Artificial Intelligence, Anhui University, Hefei, Anhui, 230601, China.

Email address: cysun@seu.edu.cn; cys@ustb.edu.cn (C. Sun).

degradation of previous policies and a reduced success rate for the overall task. It is noted that robotic manipulation tasks (e.g., pick-and-place sequences) are typically long-horizon, continuous control problems in which small parameter drifts can accumulate into large execution errors. This makes the system more sensitive to forgetting than in many other DRL domains such as video games or navigation.

Some methods have been proposed to mitigate catastrophic forgetting in sequential task learning. For example, Rusu et al. introduced specialized subnetworks to preserve knowledge across tasks, but the scalability remains limited as its parameters grow linearly with task quantity [13]. In [14], Kirkpatrick et al. proposed elastic weight consolidation (EWC) which improves memory retention by stabilizing weights related to previous tasks, but performance degrades when tasks exhibit conflicting gradients. More recent approaches, such as gradient-based task balancing [15,16] and context-aware modular architectures [17, 18], have improved generalization, but often rely on additional task-specific contextual information, thereby increasing computational complexity. Recently, Sun et al. introduced a parameter aggregation sharing mechanism, which achieves flexible parameter sharing and efficient learning by constructing a policy subspace [19].

Building on these studies, this paper proposes a parameter-compositional DRL control algorithm for robotic arm manipulation. First, we model the process of robotic arm manipulation as a multi-task reinforcement learning problem with temporal dependencies (see Section 3 for the detailed formulation). Then, we explore a compositional structure in the parameter space by defining shared parameters and task-specific parameters, enabling a clear separation between task-agnostic and task-specific components. This separation mechanism mitigates the instability caused by gradient conflicts across tasks and facilitates rapid adaptation to new tasks, thereby enhancing long-term stability and continuous learning for robotic arm manipulation. While this work focuses on continuous robotic manipulation tasks, the proposed framework is not inherently limited to continuous action spaces. The decomposition of parameters into shared and task-specific components can in principle be applied to discrete domains as well by replacing the DRL backbone with discrete-action variants. Recent advances in critic-based DRL for discrete systems provide further insights into such extensions [20]. We therefore regard the application of KCRL to discrete control problems as a promising future direction.

The main contributions of this paper are as follows:

1. We propose a novel parameter-compositional sequential reinforcement learning framework for robotic grasping, called knowledge-compositional reinforcement learning (KCRL), which explicitly separates task-agnostic shared parameters from task-specific components. This architecture enhances the model's ability to retain previously learned skills, mitigates catastrophic forgetting and enables efficient adaptation to new grasping tasks.
2. We implement KCRL based on the soft actor-critic (SAC) algorithm. Extensive experiments demonstrate that KCRL outperforms standard DRL methods (SAC, MT-SAC) in task completion time and success rate. Notably, when the conveyor belt speed reaches 0.6 m/s, KCRL achieves an 8 % reduction in task completion time and an 11 % improvement in grasping success rate compared to baseline methods.
3. We develop a multi-scenario robotic grasping experimental platform based on the VSTC ROCR6 robotic arm and make it publicly available. This benchmark provides a valuable resource for evaluating multi-task learning algorithms in robotic manipulation, fostering further research in this domain.

The rest of this paper is organized as follows. In Section 2, we briefly survey existing literature on sequential decision-making problem. Section 3 introduces the preliminaries of reinforcement learning

and parameter sharing. Section 4 presents the proposed parameter-compositional KCRL framework. Experimental evaluations are detailed in Section 5, followed by conclusions in Section 6.

2. Related work

The robotic arm manipulation tasks, such as pick-and-place, are inherently sequential decision-making problems that require continuous interaction and accompany dynamic scenarios. This brings about significant challenges such as temporal dependency and catastrophic forgetting [21,22]. Temporal dependency demands the agent to effectively model the dynamic relationships between states and actions, which is crucial for tasks that involve continuous and precise control. Meanwhile, catastrophic forgetting often leads to a gradual loss of previously acquired skills when adapting to new tasks, thereby reducing learning efficiency. To enhance training efficiency, sharing network parameters across tasks has been widely used as a common approach [23,24], enabling knowledge transfer across different tasks. However, fully shared parameter architectures frequently experience gradient conflicts between tasks, causing negative interference and impacting model generalization. To mitigate gradient conflicts, several studies have introduced parameter-constraint methods that maintain a degree of independence between tasks. For example, hypernetwork-based methods [25] employ fixed-capacity networks to model task dynamics, allowing adaptation to non-stationary environments while avoiding uncontrolled parameter growth. Similarly, learning without forgetting (LwF) constrains output distributions to ensure that acquiring new tasks does not significantly disrupt previously learned task knowledge, thus effectively mitigating forgetting [26]. Additionally, the average gradient episodic memory (A-GEM) method stores gradient information from previous tasks and applies constraints during parameter updates, maintaining a dynamic balance across multiple tasks by limiting deviations caused by learning new tasks [27].

In addition to DRL-based methods, numerous works in control theory have studied robustness under uncertainty and non-Gaussian disturbances. For example, Zhu et al. [28] proposed secure observer-based collision-free control for autonomous vehicles under non-Gaussian noise. Zhang et al. [29] proposed two adaptive strategies to address the kinematic and dynamic uncertainties of robotic manipulators, where the first enables real-time adaptation and estimation of the unknown Jacobian matrix and the second employs an observer-based estimation law. To address the robust control problem of networked systems with unknown nonlinearities, Zhao et al. [30] proposed an adaptive control strategy that integrates fuzzy approximation, state observation, and periodic event-triggering. Although these works focus on control-theoretic formulations rather than continual learning, they highlight a common challenge, maintaining robustness against uncertainty and disturbances, which is also central to robotic manipulation in our setting.

Despite their effectiveness, parameter-constraint strategies still suffer from limited adaptability due to their reliance on shared parameters. To address this limitation, researchers have proposed dynamic task-selection strategies, enabling more flexible and adaptive task modeling. For instance, routing networks [31] dynamically adjust the network's functional pathways based on task-specific requirements, significantly reducing task interference and improving learning efficiency. Similarly, attention-based mixture of experts soft actor-critic (AMESAC) [32] leverages attention mechanisms to model task relevance, allowing the model to prioritize information pertinent to the current task while suppressing the influence of irrelevant tasks. Building upon these efforts, recent approaches have introduced parameter-separation modules, providing more fine-grained policies with improved efficiency. For instance, language embedding-based generative incremental off-policy reinforcement learning framework with Non-parametric Bayes (LEGION) dynamically generates and reuses knowledge components for new tasks by leveraging language embeddings combined with a nonparametric

Bayesian model to achieve sustained adaptation [33]. Similarly, meta-reinforcement learning with task self-discovery (MELTS) uses Bayesian nonparametric techniques for self-discovery and clustering of subtasks in the latent space, significantly enhancing task adaptability and learning efficiency [34]. In contrast to existing approaches, our proposed framework defines shared and task-specific parameters to separate task-agnostic and task-specific knowledge. The shared parameters are responsible for capturing task-agnostic knowledge that can be transferred across tasks, while the task-specific parameters focus on the unique aspects of each task. This compositional parameter structure effectively reduces gradient conflicts and significantly enhances adaptability, thereby ensuring continual learning without additional task-specific constraints.

3. Preliminaries

3.1. Markov decision process

In this paper, we formulate the process of robotic arm manipulation as a Markov decision process (MDP), defined by a tuple (S, A, P, R, γ) , where:

- S denotes the set of possible states (state space).
- A denotes the set of possible actions (action space).
- $P(s'|s, a)$ represents the state transition probability, describing the probability of transitioning to state s' given the current state s and action a .
- $R(s, a)$ is the reward function, indicating the immediate reward received after taking action a in state s .
- $\gamma \in (0, 1)$ is the discount factor, balancing immediate rewards against future rewards.

An MDP characterizes a stochastic process in which the current state is determined solely by the previous state and action. For the sequentially structured pick-and-place task, we model the entire process as a single MDP, whose state space inherently encodes the progress of preceding subtasks (e.g., whether the object has been reached or grasped), with each subtask corresponding to a distinct region of the state-action space.

The goal of an MDP is to determine an optimal policy $\pi(a|s)$ that maximizes the expected cumulative reward. Specifically, the expected cumulative discounted reward under a policy π is defined as:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (1)$$

where $\mathbb{E}$ denotes the expectation operator over the trajectory distribution, and π represents the stochastic policy parameterized by θ . Consequently, the robotic arm aims to learn an optimal policy $\pi(a|s)$ that maximizes this cumulative discounted reward in expectation.

3.2. Soft actor-critic algorithm

Sequential robotic manipulation tasks involve complex temporal dependencies, requiring an algorithm that effectively balances exploration and exploitation. In this paper, we propose a sequential multi-task reinforcement learning algorithm for robotic grasping, building upon the soft actor-critic algorithm. SAC is an off-policy actor-critic method featuring entropy regularization, which encourages exploration in states of high uncertainty by maintaining an optimal trade-off between exploration and exploitation, thereby enhancing overall learning efficiency. The entropy of the policy in state s is defined as:

$$H_{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\log(\pi(a|s))] \quad (2)$$

where $a \sim \pi(\cdot|s)$ denotes that the action $a \in A$ is sampled according to the probability distribution $\pi(\cdot|s)$ over actions given state $s \in S$.

The SAC algorithm simultaneously learns a policy and two value functions during optimization. Specifically, the policy is optimized by

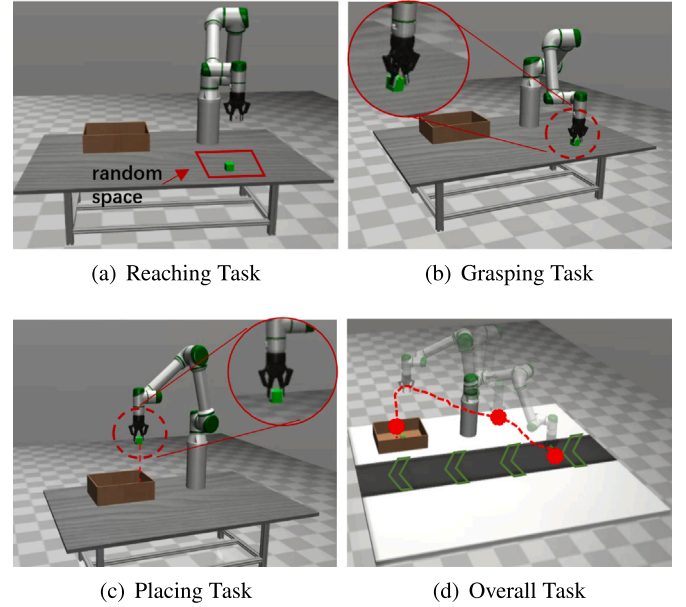


Fig. 1. The tasks are carried out in sequence, and each task depends on the parameter adjustment of the previous tasks. The first two tasks (a) and (b) focus on static target positioning and grasping, by adjusting the target position and grasping strategy as the basis; the third task (c) further considers the stability and placement accuracy of the object; The final task (d) integrates the parameters of the first three tasks and performs dynamic transport, showing the gradual optimization and dependence of parameter combinations on task effects.

maximizing the following objective function:

$$J(\pi) = \mathbb{E}_{s_t, r_t \sim D} \left[\sum_{t=0}^{\infty} (\gamma^t r_t + \alpha H_{\pi}(s_t)) \right], \quad (3)$$

where α is the entropy regularization coefficient, and r_t is the reward achieved at time step t . In SAC, the policy and values are approximated by an actor network and a critic network, respectively. The critic network is trained by minimizing the mean squared TD-errors [35].

4. Knowledge-compositional reinforcement learning for sequential manipulation tasks

In this work, we study sequential robotic manipulation tasks, specifically focusing on a typical pick-and-place scenario. The process of robotic arm manipulation can be formulated as a sequential decision-making problem, comprising multiple subtasks characterized by temporal dependencies and progressively increasing complexity. The robot is expected to continually acquire new skills while effectively preserving knowledge learned from previous tasks.

4.1. Problem setup

The pick-and-place manipulation task is generally decomposed into three sequential stages:

- Reaching (Fig. 1(a)): In the reaching phase, the end effector starts from a random initial pose and the objective is to move it into the vicinity of the target object.
- Grasping (Fig. 1(b)): The robotic arm executes precise grasping and firmly holds the target object. During this phase, the agent must retain spatial information gained from the reaching stage to ensure accurate alignment while dynamically adjusting the gripper's angle and aperture.
- Placing (Fig. 1(c)): The robotic arm transports the grasped object to a designated location and places it accurately. This stage significantly

increases complexity, requiring the robot to execute continuous control actions while effectively preserving critical information from prior stages to ensure overall task coherence.

Through these sequential stages, the robotic arm efficiently performs continuous pick-and-place operations, such as grasping objects moving along a conveyor belt (Fig. 1(d)), while ensuring stable knowledge retention across tasks, thereby supporting robust and autonomous robotic manipulation.

4.2. Modeling sequential manipulation tasks as MDP

For pick-and-place tasks, the robotic manipulator must continuously perceive critical environmental information to support accurate decision-making and precise action execution.

4.2.1. State space

In this work, we design a comprehensive state space that incorporates essential task-related information, with particular attention given to the robotic arm's actual operating parameters. Specifically, the state space is defined as follows:

$$\mathbf{s}_t = \langle P_{\text{gripper}}, P_{\text{cube}}, P_{\text{relative}}, R_{\text{cube}}, V_{\text{gripper}}, V_{\text{cube}}, \omega_{\text{cube}}, g, t \rangle, \quad (4)$$

where P_{gripper} and P_{cube} represent the positions of the gripper and the target object, respectively, defining their spatial relationship; P_{relative} denotes the relative positions between them; R_{cube} reflects the target object's rotation angle; V_{gripper} and V_{cube} are the linear velocities of the gripper and the object; ω_{cube} represents the angular velocity of the object; g indicates the gripper's open/closed state; and t signifies the task phase. These components form the state space, enabling the manipulator to make informed decisions and execute precise actions in a dynamic environment.

4.2.2. Action space

The six-degree-of-freedom (6-DoF) robotic manipulator, known for its kinematic flexibility and versatility, is widely used in executing complex industrial tasks such as precision grasping and object placement. In order to ensure that our model training does not exceed the capabilities of the selected six-axis robotic arm, we have carefully considered the design of the action space to ensure that it closely corresponds to the actual operational mode of the real robotic arm. The action space is formulated as a seven-dimensional continuous vector $\mathbf{a} \in \mathbb{R}^7$, representing joint-level actuation and end-effector control:

$$\mathbf{a} = [a_1, a_2, a_3, a_4, a_5, a_6, a_7]^T \in [-1.0, 1.0]^7, \quad (5)$$

Each action component is constrained within the interval $[-1.0, 1.0]$, representing incremental adjustments mapped to joint-space trajectories. The first six dimensions $a_1, a_2, a_3, a_4, a_5, a_6$ govern the kinematics of the manipulator's joints, enabling continuous reconfiguration of the end-effector within the task space. These adjustments align with the actual joint range of the real robot, typically within $[-180^\circ, 180^\circ]$ for each joint, with a maximum joint velocity of $180^\circ/\text{s}$. The seventh control command, a_7 , regulates the gripper's state, facilitating the acquisition and release of objects in the operational environment. This action space design ensures precise kinematic control, supporting complex operations such as sequential grasping, translational motion, and accurate object placement, while taking into account the hardware limitations of the robotic arm.

4.2.3. Reward function

The pick-and-place task faces the challenge that sparse reward signals cannot effectively guide the robot to complete the staged tasks of "reaching", "grasping" and "placing", and different reward modes are required at different stages. Based on this, this study designed a dense staged reward signal to provide continuous strategy guidance while maintaining task coherence, ensuring that the robot can efficiently learn complex operation sequences.

Distance-based reward: The distance-based reward is crucial for evaluating the proximity between the robotic arm, the target object, and the target location, as it provides continuous feedback on the arm's progress toward the desired goal. Specifically, two key distances are considered: the distance between the target object and the gripper (denoted as $d_{\text{cube-gripper}}$) and the distance between the target object and the target location (denoted as $d_{\text{cube-box}}$). The following position vectors are defined:

- $\mathbf{p}_c = [P_{c_x}, P_{c_y}, P_{c_z}]^T$: the position of the target object.
- $\mathbf{p}_g = [P_{g_x}, P_{g_y}, P_{g_z}]^T$: the position of the gripper's end effector.
- $\mathbf{p}_b = [P_{b_x}, P_{b_y}, P_{b_z}]^T$: the position of the target location.

The Euclidean distance between the cube and the gripper is given by:

$$d_{\text{cube-gripper}} = \|\mathbf{p}_c - \mathbf{p}_g\|_2, \quad (6)$$

where $\|\cdot\|_2$ denotes the Euclidean norm. Similarly, the Euclidean distance between the cube and the target box is:

$$d_{\text{cube-box}} = \|\mathbf{p}_g - \mathbf{p}_b\|_2, \quad (7)$$

These distances, $d_{\text{cube-gripper}}$ and $d_{\text{cube-box}}$, provide precise measures of proximity, which are essential for guiding the robotic arm's actions. Then, the distance-based reward R_{dist} is defined as:

$$R_{\text{dist}} = \begin{cases} 20 - 10 \times d_{\text{cube-box}}, & \text{if } d_{\text{cube-gripper}} < 0.02\text{m}, \\ -10 \times d_{\text{cube-gripper}}, & \text{otherwise.} \end{cases} \quad (8)$$

In the reaching phase, a penalty for large gripper-object distances guides the robotic arm to quickly locate the target. When $d_{\text{cube-gripper}} < 0.02\text{m}$, the robot switches to the transport phase, and the reward focuses on minimizing the distance between the object and the target location, ensuring accurate placement. This two-phase structure aligns closely with the sequential nature of the robotic grasping task, providing clear guidance for policy learning.

Height-based Reward: In addition to the distance-based rewards, a height-based reward R_{height} is introduced to ensure the target object is placed at the correct height in the target location. The height of the target object is represented by the z -coordinate of P_b . The height-based reward is formulated as:

$$R_{\text{height}} = 50 \times (P_{b_z} - 0.45). \quad (9)$$

Eq. (9) introduces a vertical placement reward to ensure correct object height, where the reference value 0.45 corresponds to the bin surface in our setup, and the scaling factor 50 was empirically chosen to balance the reward magnitude with other terms.

By introducing vertical constraints, positive feedback is provided when the object approaches the target height, while penalties are applied for objects falling to low heights, ensuring stability during the transport phase. The reward function comprehensively accounts for the spatial complexity of the robotic arm's task.

The total reward R_{total} of the overall task is the sum of the distance-based and height-based rewards:

$$R_{\text{total}} = R_{\text{dist}} + R_{\text{height}}, \quad (10)$$

where R_{total} considers the distances between the gripper and the target object, the target object and the target location, as well as the target object's height. This total reward effectively guides the robotic arm in pick-and-place tasks.

4.3. Knowledge compositional mechanism

To handle the sequential and multi-subtask characteristics of pick-and-place manipulation, we adopt a structured parameter-compositional

Algorithm 1 Knowledge-compositional reinforcement learning for sequential manipulation tasks.

```

1: Input: parameter-set size  $K$ , learning rate  $\lambda$ , task sequence  $\mathcal{T}$ ;
2: Initialize: shared parameter matrix  $\Phi = [\phi_1, \phi_2, \dots, \phi_K]$ ;
3: for each task  $T_i$  in sequence  $\mathcal{T}$  do
4:   Initialize task policy  $\pi_\tau$  and task embedding  $w_\tau$ ;
5: end for
6: for episode = 1 to max_episodes do
7:   Sample a task  $T_i$  from  $\mathcal{T}$ ;
8:   Initialize state  $s$ ;
9:   for t = 1 to max_steps do
10:    Compute task parameters:  $\theta_\tau = \Phi w_\tau$ ;
11:    Select action  $a_t = \pi_{\theta_\tau}(s_t)$ ;
12:    Execute action and observe  $(r_t, s_{t+1})$ ;
13:    Store transition  $(s_t, a_t, r_t, s_{t+1})$ ;
14:     $s_t \leftarrow s_{t+1}$ ;
15:  end for
16:  Sample a batch of transition data from replay buffer.
17:  Calculate the loss function in Eq. (12).
18:  Calculate the regularization term in Eq. (13).
19:  Update  $\phi$  and  $w_i$  via back propagation;
20:  if task  $T_i$  is completed then
21:    Switch to next task in  $\mathcal{T}$ ;
22:  end if
23: end for

```

architecture that explicitly separates parameters into task-agnostic and task-specific components, thereby mitigating catastrophic forgetting and reducing gradient conflicts common in fully shared networks. Specifically, we consider pick-and-place manipulation task $\mathcal{T}$ with N tasks, denoted as $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$ in sequence. Each task T_i corresponds to a policy $\pi_{\theta_i}(a|s)$, $i = 1, \dots, N$. At each time step t , the agent focuses on a single subtask T_i and selects actions using the policy $\pi_{\theta_i}(a|s)$, based on the current state s_t . Different from the full-shared network, our policy $\pi_{\theta_i}(a|s)$ combines shared (task-agnostic) and task-specific parameters, and is defined as:

$$\theta_i = g(\theta_{\text{shared}}^{(i-1)}, \theta_{\text{task-specific}}^{(i)}), i = 1, \dots, N,$$

where $\theta_{\text{shared}}^{(i-1)}$ denotes the parameters shared¹ across tasks up to T_{i-1} , and $\theta_{\text{task-specific}}^{(i)}$ represents those unique to task T_i . The composition function $g(\cdot)$ integrates these two components to form the policy parameters for task T_i .

In this paper, we follow the setup in [19]. The shared parameters are represented by a matrix consisting of a set of K parameters, e.g., $\theta_{\text{shared}}^{(i-1)} = \Phi = [\phi_1, \phi_2, \dots, \phi_K]^T \in \mathbb{R}^{K \times m}$. The task-specific parameters are represented by a K -dimensional vector (i.e., $\theta_{\text{task-specific}}^{(i)} = w_i \in \mathbb{R}^K$), which is generated by a task-encoding network. Accordingly, the policy parameters for task T_i (i.e., $\pi_{\theta_i}(a|s)$) can be expressed as:

$$\theta_i = g(\Phi, w_i) = \Phi^T w_i. \quad (11)$$

To optimize the model across multiple tasks of sequential manipulation tasks, where each task builds upon the previous one, we employ a multi-task loss function. Each subtask T_i (e.g., reaching, grasping, or placing) is associated with a specific loss $L_i(\theta_i)$, and the overall loss is formulated as a sum of these individual losses:

$$L(\theta) = \sum_{i=1}^N L_i(\theta_i) = \sum_{i=1}^N L_i(g(\Phi, w_i)), \quad (12)$$

¹ The superscript (i) indicates the stage of training (i.e., after learning task T_i), but the parameters remain shared across all tasks. This notation is used only to distinguish the evolving states of the shared parameter set.

where L_i is the loss of task T_i . Then, we have

$$\nabla_{\Phi} L = \sum_{i=1}^N \nabla_{\Phi} L_i, \quad (\text{task-shared})$$

$$\nabla_{w_i} L = \nabla_{w_i} L_i. \quad (\text{task-specific})$$

This decomposition mitigates catastrophic forgetting because new tasks mainly update their task-specific coefficients without overwriting the shared basis, thereby preserving previously learned knowledge. Moreover, the composition reduces gradient conflicts by orthogonalizing task-specific updates: gradients from different tasks interact through their respective w_i vectors rather than directly in a holistic parameter space.

By contrast, fully shared networks force all tasks to update the same set of parameters, so gradient directions from different tasks often conflict, leading to instability and overwriting of prior knowledge. Different from that, KCRL's composition scheme ensures that shared parameters evolve smoothly while task-specific branches absorb task-dependent variations. This provides a principled balance between stability (retaining prior tasks) and plasticity (adapting to new tasks).

When updating from $\theta_{\text{shared}}^{(i-1)}$ to $\theta_{\text{shared}}^{(i)}$, the change $\Delta\theta = \theta_{\text{shared}}^{(i)} - \theta_{\text{shared}}^{(i-1)}$ may cause a loss increase on previously learned tasks. Under the common smoothness assumption of the loss function, this increase is bounded by a quadratic term proportional to $\|\Delta\theta\|^2$ [36]. To penalize this deviation, thereby constraining the drift of shared parameters and mitigating catastrophic forgetting, we introduce the following regularization term:

$$\Omega(\theta_{\text{shared}}) = \lambda \|\theta_{\text{shared}}^{(i)} - \theta_{\text{shared}}^{(i-1)}\|^2, \quad (13)$$

where λ is the regularization hyperparameter that controls the strength of the regularization. Including Eq. (13) in the objective leads to an effective update condition: $\nabla L_i(\theta_{\text{shared}}^{(i)}) + 2\lambda(\theta_{\text{shared}}^{(i)} - \theta_{\text{shared}}^{(i-1)}) = 0$. Here the additional term acts as a restoring force that keeps $\theta_{\text{shared}}^{(i-1)}$ close to $\theta_{\text{shared}}^{(i)}$. This reduces gradient interference across tasks and promotes stability, while still allowing plasticity through task-specific parameters $\theta_{\text{task-specific}}^{(i)}$.

Remark 1. Eq. (13) can be interpreted as a proximal-style constraint, in line with approaches commonly used in continual learning (e.g., EWC [14], proximal regularization). For simplicity and computational efficiency in robotic control loops, we adopt a uniform penalty, while noting that the formulation can naturally be extended to importance-weighted variants.

Since subtasks exhibit temporal dependencies, each builds upon the successful learning of the preceding one. Therefore, stable and efficient training is essential. To this end, we adopt a twofold task-switching criterion: (1) the accumulated reward of the current subtask must exceed a task-dependent threshold to guarantee adequate performance, and (2) the relative change in cumulative rewards within a fixed window (e.g., 1000 steps) must fall below a predefined margin δ (e.g., 5 %) to indicate stability. Only when both conditions are satisfied does the agent transition to learning the subsequent subtask.

Based on the above preparation, we extend the SAC algorithm with a structured parameter-compositional architecture to address the limitations of standard DRL frameworks in capturing interdependencies and facilitating knowledge transfer across subtasks. In our implementation, the actor and critic networks in the original SAC are replaced with compositional fully connected layers that consist of K parameter sets shared across tasks. The key difference is that in KCRL, both the actor and critic additionally take task-specific variables as input. These task-specific variables are generated by a standard MLP with two fully-connected layers that take the observations as input. The overall framework and the pseudocode of the proposed algorithm are illustrated in Fig. 2 and Algorithm 1, respectively.

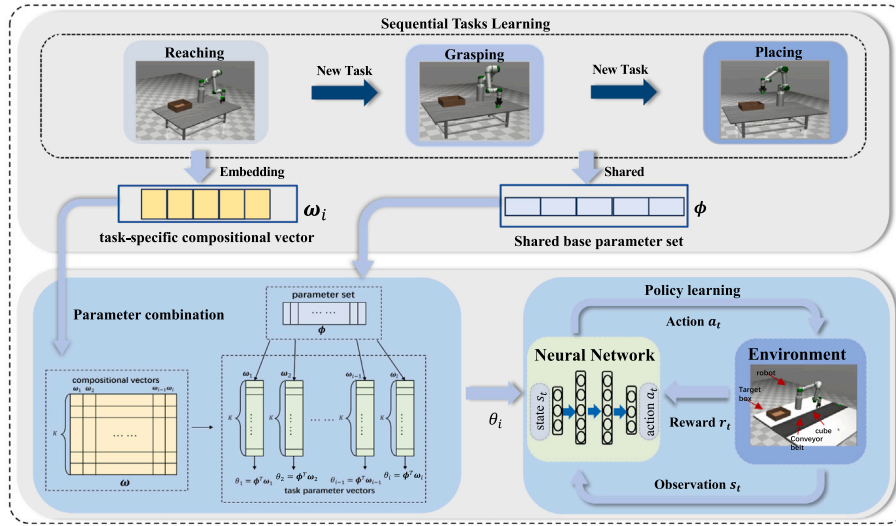


Fig. 2. Overview of the sequential task reinforcement learning framework.

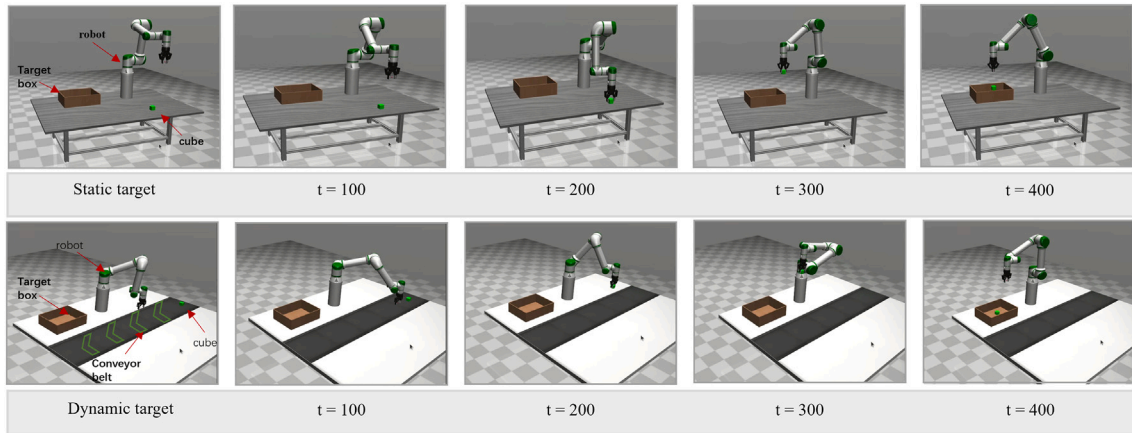


Fig. 3. The process of the robotic arm performing the pick-and-place task under both static and moving conveyor. Each scene depicts the state of the robotic arm at different steps (100, 200, 300, and 400 steps).

5. Experiment and analysis

5.1. Environment setup

We conduct all experiments on the MuJoCo simulation platform, which provides high-fidelity dynamic simulation of both the robotic arm and the conveyor system (Fig. 3). Our setup employs a VSTC ROCR6 (6-DOF) robotic arm, paired with a Robotiq 85 gripper to perform precise pick-and-place manipulation tasks. By adding a moving conveyor belt system, we introduce continuous object motion, which increases task complexity and dynamics, and provides a more rigorous evaluation of algorithmic adaptability.

The pick-and-place experiment is divided into three sequential phases: object reaching (Fig. 1(a)), object grasping (Fig. 1(b)), and object placement (Fig. 1(c)). The difficulty increases progressively, mirroring the sequential learning process of robotic manipulation. In this process, the agent first learns simple operations and retains previous knowledge as it gradually masters more complex tasks. To evaluate the algorithm's performance and adaptability under varying dynamic conditions, we additionally perform a conveyor speed generalization experiment across a range of belt speeds, thereby testing its robustness and efficiency in dynamic environments.

We note that our experimental setup introduces uncertainties such as randomized object positions, variable motion speeds, and dynamic disturbances. These scenarios are designed to approximate the non-Gaussian noise and uncertain dynamics. While our approach is reinforcement learning-based, the challenges addressed are aligned: ensuring robustness to uncertainty and variability remains a shared goal across both control theory and learning-based frameworks.

5.2. Baselines and metrics

To evaluate the effectiveness of our robotic arm manipulation system, we employ three key metrics: (1) learning efficiency, measured by the rate of reward improvement over training episodes and indicative of the algorithm's adaptability and convergence speed; (2) task success rate, defined as the proportion of trials in which the agent successfully reaches, grasps and places the object, directly reflecting control robustness; (3) task completion time, capturing both system responsiveness and potential bottlenecks in dynamic environments.

We compare our method against two baselines: SAC [35], which trains independent agents for each task, and Multi-Task SAC (MT-SAC) [37], which incorporates task identifiers via one-hot encoding to

Table 1
Summary of hyperparameters used in experiments.

Hyperparameter	Value	Notes
Learning rate	3×10^{-4}	For both actor and critic (Adam)
Batch size	256	Sampled from replay buffer
Discount factor γ	0.99	Standard in continuous control
Entropy temperature α	0.2	Tuned for balance of exploration
Number of parameter sets K	5	Scale of parameter sets, shared across tasks
Hidden layers	2×256	Two fully-connected layers per network
Replay buffer size	10^6	Shared across tasks
Update frequency	Every 1 step	One gradient update per env step
Target smoothing τ	0.005	For soft target updates

enable multi-task learning. While MT-SAC improves sample efficiency compared to SAC, it does not explicitly model parameter composition, potentially limiting generalization across sequential tasks. In contrast, our approach leverages shared and task-specific parameters, enabling efficient knowledge transfer and adaptation to dynamic conditions. More recently, parameter composition (PACO) [19] was introduced to address multi-task DRL, which is also built on SAC framework. We include PACO as another baseline in our experiments for a fair comparison.

5.3. Training setup

To ensure clarity and reproducibility, the hyperparameter settings used in our experiments are summarized in Table 1. The values were selected based on common practice in DRL and further tuned within a reasonable range to stabilize training. All experiments were conducted on a workstation equipped with an AMD Ryzen Threadripper PRO 5975WX 32-core CPU and 512 GB of RAM, and an NVIDIA GeForce RTX 4090 GPU with 24 GB of memory for deep learning computations.

5.4. Pick-and-place with static objects

To evaluate the effectiveness of our approach in sequential robotic manipulation, we perform experiments in a static setting without conveyor belt motion. In the static pick-and-place task, the reaching task ensures that the robotic arm is accurately positioned near the target object, laying the foundation for subsequent tasks. The grasping task evaluates the robotic arm's ability to pick up objects from random positions, requiring not only precise positioning but also fine control of the gripper's opening size and angle. The placing task assesses the robotic arm's ability to accurately position and release the grasped object at a designated location, requiring coordinated motion planning and precise spatial awareness. This suggests that among sequential tasks, some knowledge is task-agnostic while other knowledge is task-specific.

We focus our experiments on the latter two tasks: grasping and placing. Both tasks are based on the reaching task, and the placing task further builds upon the grasping task. All models are initially trained using SAC on the reaching task. We then evaluate and compare KCRL with baseline algorithms in the subsequent grasping and placing phases.

As shown in Fig. 4(a), while maintaining the original reaching task strategy, all three algorithms demonstrate steadily increasing rewards in training progress. Compared to the other algorithms, KCRL achieves a faster reward increase and higher rewards, indicating that it better retains the original reaching skills and is more efficient in learning of the grasping task. PACO achieves faster early-stage learning compared with SAC, but its final performance remains lower than MT-SAC and our proposed KCRL. As the difficulty of the sequential tasks increases, performance differences between algorithms become more evident. As shown in Fig. 4(b), PACO suffers from large performance variance and slower convergence, highlighting its difficulty in handling sequential dependencies. In contrast, KCRL significantly outperforms the other two algorithms, benefiting from the shared parameters that transmit common knowledge and reduce catastrophic forgetting, resulting in faster convergence.

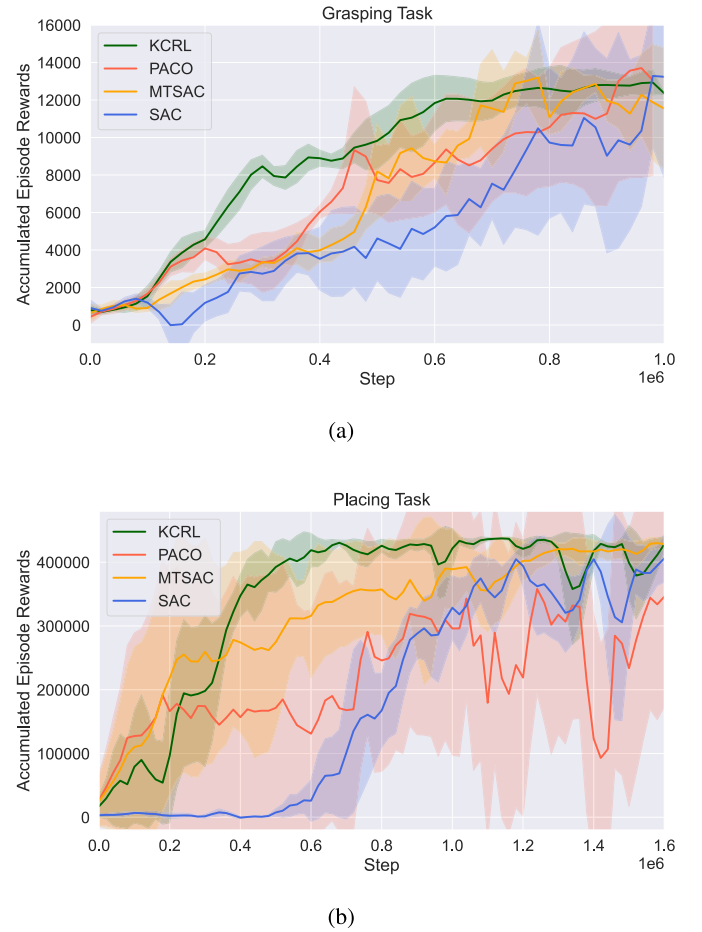


Fig. 4. Cumulative rewards in basic tasks: (a) Grasping task; (b) Placing task.

To further evaluate the performance of our algorithm, we conduct a 1000-episode test on the trained models in the previously mentioned scenarios, as shown in Fig. 3. We compare the average task completion time and success rate across different algorithms, as presented in Table 2. The average time is calculated over both successful and failed episodes, with failed cases assigned the maximum time horizon. Vanilla SAC generally requires more corrective steps, leading to longer episode durations compared to compositional variants. The results show that for grasping tasks, KCRL and MT-SAC perform comparably; however, when transitioning to the subsequent tasks, KCRL outperforms the baselines, maintaining a 100 % success rate.

5.5. Pick-and-place with moving objects

In real-world industrial settings, robotic arms often operate under dynamic conditions where objects move at varying speeds. This introduces challenges such as varying response times and precision requirements for grasping. To evaluate the algorithm's generalization performance under such conditions, we designed a conveyor belt grasping scenario, as shown in Fig. 3, and conducted experiments at three conveyor speeds of 0.6 m/s, 0.9 m/s, and 1.2 m/s.

As shown in Fig. 5, KCRL consistently outperforms the baseline algorithm, achieving faster convergence and higher cumulative rewards across all speed settings. At 0.6 m/s, KCRL demonstrates strong adaptability, achieving a 98 % success rate with an average completion time of 2.576 s (Table 2), surpassing PACO (94 %, 2.691 s) MT-SAC (87 %, 2.819 s) and SAC (73 %, 2.963 s). This indicates that even in relatively simple dynamic environments, KCRL provides more accurate and efficient grasping. As task complexity increases, particularly at 0.9 m/s due

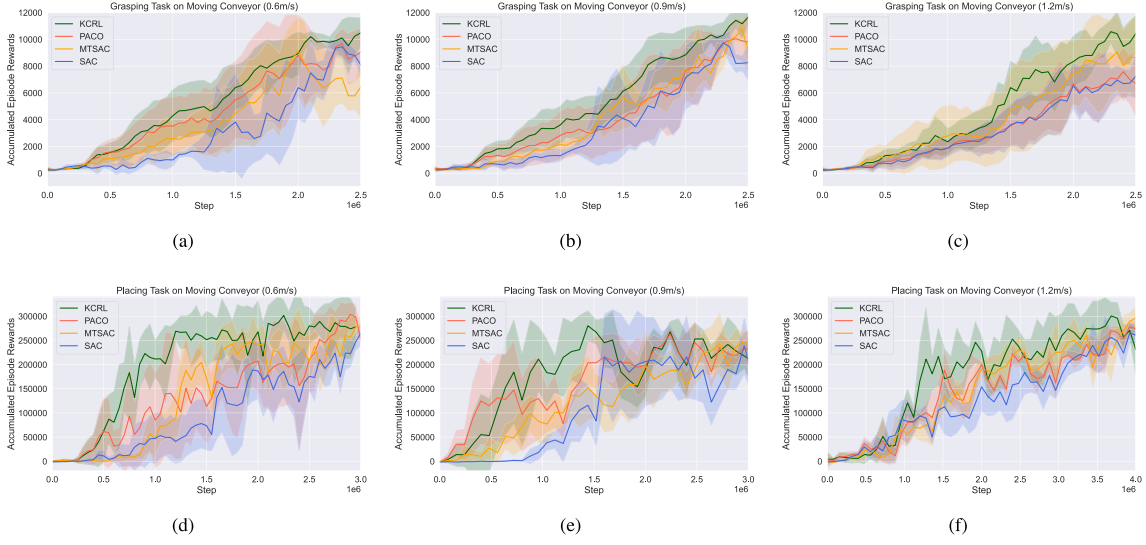


Fig. 5. Accumulative reward curves during training for pick-and-place tasks on moving conveyor: (a) Grasping task on moving conveyor (0.6 m/s); (b) Grasping task on moving conveyor (0.9 m/s); (c) Grasping task on moving conveyor (1.2 m/s); (d) Placing task on moving conveyor (0.6 m/s); (e) Placing task on moving conveyor (0.9 m/s); (f) Placing task on moving conveyor (1.2 m/s).

Table 2

Task completion times and success rates.

Task	Algorithm	Average Time (s)	Success Rate (%)
Static object	KCRL (Ours)	1.983	100 %
	PACO	3.121	84 %
	MT-SAC	2.247	92 %
	SAC	2.795	87 %
Moving object (0.6 m/s)	KCRL (Ours)	2.576	98 %
	PACO	2.691	94 %
	MT-SAC	2.819	87 %
	SAC	2.963	73 %
Moving object (0.9 m/s)	KCRL (Ours)	2.616	93 %
	PACO	2.521	90 %
	MT-SAC	2.437	84 %
	SAC	2.835	72 %
Moving object (1.2 m/s)	KCRL (Ours)	1.683	83 %
	PACO	1.703	81 %
	MT-SAC	1.756	79 %
	SAC	1.965	66 %

to the shorter response time, the performance gap between KCRL and the baselines becomes more pronounced. This highlights KCRL's robustness and adaptability under increased motion speeds. At 1.2 m/s, the rapid movement of objects significantly reduces the grasping window, making the task more challenging. However, KCRL still maintains strong performance, achieving an 83 % success rate with a completion time of 1.683 s (Fig. 5(f)). In contrast, PACO, MT-SAC and SAC achieve 81 % (1.703 s), 79 % (1.756 s) and 66 % (1.965 s), respectively. These results demonstrate that compositional architecture of KCRL enables superior generalization and responsiveness in high-speed manipulation scenarios. In contrast, the success rates of PACO, MT-SAC and SAC drop more significantly because they lack effective mechanisms to handle such dynamic scenarios. Interestingly, performance at 1.2 m/s is slightly higher than at 0.9 m/s. We conjecture that this is because once the object is grasped, the higher speed brings it closer to the target box, effectively reducing the placement distance and offsetting the increased grasping difficulty.

These results validate KCRL's generalization performance across varying speed conditions and its effectiveness in cross-task knowledge transfer. As summarized in Table 2, KCRL remains highly stable in low

and medium-speed settings while maintaining competitive performance in high-speed scenarios, with room for further optimization. The advantage of our algorithm lies in its ability to adaptively adjust the shared and task-specific parameters during task transitions. By effectively utilizing prior knowledge, it enhances training efficiency and the generalization of the policy, ensuring stable performance across different task conditions.

5.6. Forgetting score

In this paper, we adopt a standard forgetting score defined as

$$F^{(i)} = \max G_{\text{prev}}^{(i-1)} - G_{\text{end}}^{(i-1)}. \quad (14)$$

where $\max G_{\text{prev}}^{(i-1)}$ is the cumulative reward on prior tasks (T_{i-1}) trained previously, and $G_{\text{end}}^{(i-1)}$ is the final performance for prior tasks after learning a new task (T_i).

Fig. 6 illustrates the forgetting score of the Grasping task during the training of the Placing task. We observe a characteristic three-phase dynamic. At the beginning of training, the forgetting score remains low because the model has not yet substantially altered the shared parameters, and thus the performance on the Grasping task is largely preserved. As training progresses (around 0.25×10^6 steps), the forgetting score peaks sharply. This corresponds to the rapid adaptation phase of the Placing task, during which intensive updates to shared parameters overwrite previously acquired knowledge, resulting in the most severe catastrophic forgetting. Subsequently, the forgetting score gradually decreases as the Placing task converges and the updates to shared parameters stabilize. Notably, KCRL achieves both the lowest peak and the fastest decline, demonstrating its ability to mitigate abrupt performance deterioration and to stabilize knowledge retention compared to the baselines.

5.7. Ablation study

To assess the contribution of the compositional parameter design, we conducted an ablation study by varying the regularization factor λ in Eq. (13). Fig. 7 illustrates that when $\lambda = 0$ (no compositional constraint), the training process is less stable and converges more slowly. In contrast, larger λ values (e.g., 0.6 and 1.0) lead to faster convergence and higher final rewards, confirming that the compositional design plays a critical role in improving both stability and performance.

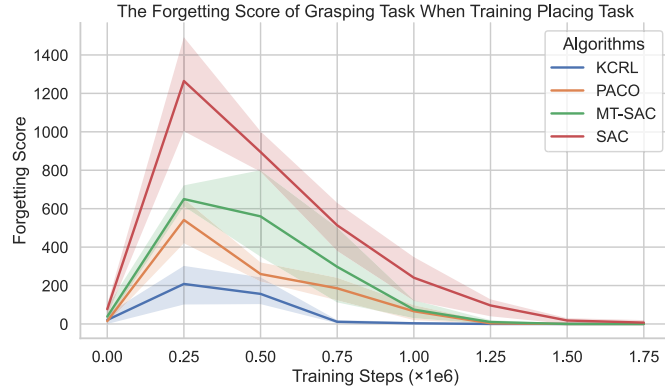


Fig. 6. The forgetting score of Grasping task during the training of Placing task.

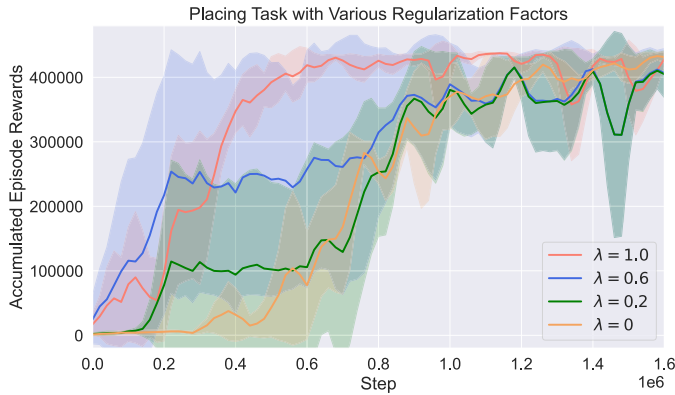


Fig. 7. Comparisons of the performance with various regularization factor λ .

5.8. Future work

Although the present evaluation is conducted in MuJoCo simulation, the environments have been carefully constructed according to the specifications of real six-axis robotic manipulator. Building on this foundation, our ongoing work is focused on deploying the proposed KCRL framework on the physical platform. This involves hardware integration, communication interface configuration, and safety validation, etc. We consider real-robot validation an essential next step, and we expect that these experiments will provide practical confirmation of the method's robustness and adaptability. In addition, we recognize that our current evaluation is limited to the three-stage pick-and-place pipeline, and extending KCRL to more complex long-horizon task sequences remains an important direction for future study.

6. Conclusions

In this paper, we propose a parameter-composition-based reinforcement learning algorithm for robotic arm manipulation in sequential tasks. By formulating the process of robotic arm manipulation as a sequential reinforcement learning problem with temporal dependencies, our approach effectively captures inter-task relationships and improves learning efficiency. The algorithm leverages a parameter-separation structure, where shared parameters transfer generalizable knowledge while task-specific parameters enable adaptability to new tasks. This design mitigates learning instability caused by cross-task gradient conflicts and enhances continual learning capabilities. Experiments conducted in a simulated robotic environment demonstrate that our method outperforms standard deep reinforcement learning baselines in terms of efficiency, success rate, and adaptability to dynamic conditions. These

results validate the effectiveness of our approach in solving complex sequential manipulation tasks by robotic arm control. In future work, we plan to deploy our approach on real robotic systems to further evaluate its performance in real-world applications, bridging the gap between simulation and practice.

CRedit authorship contribution statement

Wenzhang Liu: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Wanyi Yao:** Writing – review & editing, Writing – original draft, Visualization, Validation. **Keke Yang:** Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Lu Ren:** Writing – review & editing, Resources, Methodology, Funding acquisition, Formal analysis. **Changyin Sun:** Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Lu Ren reports that financial support was provided by National Natural Science Foundation of China. Changyin Sun reports that financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Nos. 62303009, 62236002, and 62495083).

Data availability

Data will be made available on request.

References

- [1] E. Triantafyllidis, F. Acero, Z. Liu, Z. Li, Hybrid hierarchical learning for solving complex sequential tasks using the robotic manipulation network roman, *Nat. Mach. Intell.* 5 (2023) 991–1005.
- [2] J. Huang, X. Tu, J. He, Design and evaluation of the rupert wearable upper extremity exoskeleton robot for clinical and in-home therapies, *IEEE Trans. Syst. Man Cybern. Syst.* 46 (2016) 926–935.
- [3] S. Kleff, A. Meduri, R. Budhiraja, N. Mansard, L. Righetti, High-frequency nonlinear model predictive control of a manipulator, in: 2021 IEEE International Conference on Robotics and Automation, 2021, pp. 7330–7336.
- [4] F.-J. Chu, R. Xu, P.A. Vela, Real-world multiobject, multigrasp detection, *IEEE Robot. Autom. Lett.* 3 (2018) 3355–3362.
- [5] G. Du, K. Wang, S. Lian, K. Zhao, Vision-based robotic grasping from object localization, object pose estimation to grasp estimation for parallel grippers: a review, *Artif. Intell. Rev.* 54 (2021) 1677–1734.
- [6] H.-S. Fang, C. Wang, M. Gou, C. Lu, Graspnet-1billion: a large-scale benchmark for general object grasping, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 11444–11453.
- [7] Y. Ren, Z. Zhou, Z. Xu, Y. Yang, G. Zhai, M. Leibold, F. Ni, Z. Zhang, M. Buss, Y. Zheng, Enabling versatility and dexterity of the dual-arm manipulators: a general framework toward universal cooperative manipulation, *IEEE Trans. Robot.* (2024).
- [8] X. He, Z. Hu, H. Yang, C. Lv, Personalized robotic control via constrained multi-objective reinforcement learning, *Neurocomputing* 565 (2024) 126986.
- [9] C. Ma, J. Zhang, J. Liu, L. Ji, F. Gao, A parallel multi-module deep reinforcement learning algorithm for stock trading, *Neurocomputing* 449 (2021) 290–302.
- [10] Y. Lin, J. Huang, M. Zimmer, Y. Guan, J. Rojas, P. Weng, Invariant transform experience replay: data augmentation for deep reinforcement learning, *IEEE Robot. Autom. Lett.* 5 (2020) 6615–6622.
- [11] X. Yu, R. Shi, P. Feng, Y. Tian, S. Li, S. Liao, W. Wu, Leveraging partial symmetry for multi-agent reinforcement learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024, pp. 17583–17590.
- [12] J. Zhang, M. Wu, H. Dong, Generative category-level object pose estimation via diffusion models, in: Advances in Neural Information Processing Systems, vol. 36, 2024a.
- [13] A.A. Rusu, N.C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, R. Hadsell, Progressive neural networks, *arXiv preprint arXiv:1606.04671* (2016).

- [14] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A.A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al., Overcoming catastrophic forgetting in neural networks, *Proc. Natl. Acad. Sci.* 114 (2017) 3521–3526.
- [15] X. Chen, C. Gong, Q. Wan, L. Deng, Y. Wan, Y. Liu, B. Chen, J. Liu, Transfer learning for deep neural network-based partial differential equations solving, *Adv. Aerodyn.* 3 (2021) 1–14.
- [16] S. Goswami, K. Kontolati, M.D. Shields, G.E. Karniadakis, Deep transfer operator learning for partial differential equations under conditional shift, *Nat. Mach. Intell.* 4 (2022) 1155–1164.
- [17] S. Sodhani, A. Zhang, J. Pineau, Multi-task reinforcement learning with context-based representations, in: *International Conference on Machine Learning*, 2021, pp. 9767–9779.
- [18] X. Liu, J. Wu, S. Chen, A context-based meta-reinforcement learning approach to efficient hyperparameter optimization, *Neurocomputing* 478 (2022) 89–103.
- [19] L. Sun, H. Zhang, W. Xu, M. Tomizuka, Paco: parameter-compositional multi-task reinforcement learning, in: *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 21495–21507.
- [20] X. Wang, D. Ding, X. Ge, Q.-L. Han, Supplementary control for quantized discrete-time nonlinear systems under goal representation heuristic dynamic programming, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (3) (2022) 3202–3214.
- [21] A. Xie, C. Finn, Lifelong robotic reinforcement learning by retaining experiences, in: *Conference on Lifelong Learning Agents*, 2022, pp. 838–855.
- [22] A. Chen, A. Sharma, S. Levine, C. Finn, You only live once: single-life reinforcement learning, in: *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 14784–14797.
- [23] E. Parisotto, J.L. Ba, R. Salakhutdinov, Actor-mimic: deep multitask and transfer reinforcement learning, in: *4th International Conference on Learning Representations*, 2016.
- [24] K.R. Zentner, U. Puri, Y. Zhang, R. Julian, G.S. Sukhatme, Efficient multi-task learning via iterated single-task transfer, in: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2022, pp. 10141–10146.
- [25] Y. Huang, K. Xie, H. Bharadhwaj, F. Shkurti, Continual model-based reinforcement learning with hypernetworks, in: *2021 IEEE International Conference on Robotics and Automation*, 2021, pp. 799–805.
- [26] Z. Li, D. Hoiem, Learning without forgetting, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (2017) 2935–2947.
- [27] A. Chaudhry, M. Ranzato, M. Rohrbach, M. Elhoseiny, Efficient lifelong learning with A-GEM, in: *7th International Conference on Learning Representations*, 2019.
- [28] K. Zhu, Z. Wang, Z. Li, C.-Z. Xu, Secure observer-based collision-free control for autonomous vehicles under non-gaussian noises, *IEEE Trans. Ind. Inf.* (2024).
- [29] M. Zhang, Z. Zhang, M. Sun, Adaptive tracking control of uncertain robotic manipulators, *IEEE Trans. Circuits Syst. II Express Briefs* 71 (5) (2024b) 2734–2738.
- [30] N. Zhao, H. Zhang, X. Qiu, R.K. Agarwal, Observer-based periodic event-triggered adaptive fuzzy control for networked nonlinear systems, *IEEE Trans. Cybern.* (2025).
- [31] C. Rosenbaum, T. Klinger, M. Miemer, Routing networks: adaptive selection of nonlinear functions for multi-task learning, *6th International Conference On Learning Representations* (2018).
- [32] G. Cheng, L. Dong, W. Cai, C. Sun, Multi-task reinforcement learning with attention-based mixture of experts, *IEEE Robot. Autom. Lett.* 8 (2023) 3812–3819.
- [33] Y. Meng, Z. Bing, X. Yao, K. Chen, K. Huang, Y. Gao, F. Sun, A. Knoll, Preserving and combining knowledge in robotic lifelong reinforcement learning, *Nat. Mach. Intell.* (2025) 1–14.
- [34] Z. Bing, Y. Yun, K. Huang, A. Knoll, Context-based meta-reinforcement learning with bayesian nonparametric models, *IEEE Trans. Pattern Anal. Mach. Intell.* (2024).
- [35] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International Conference on Machine Learning*, 2018, pp. 1861–1870.
- [36] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, vol. 87, Springer Science & Business Media, 2013.

- [37] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, S. Levine, Meta-world: a benchmark and evaluation for multi-task and meta reinforcement learning, in: *Conference on Robot Learning*, 2020, pp. 1094–1100.

Author biography



Wenzhang Liu received the B.S degree in department of control science and engineering in Jilin University, Changchun, China in 2016 and the Ph.D. degree in School of Automation in Southeast University, Nanjing, China in 2022, respectively. He is currently working as a Lecture at School of Artificial Intelligence, Anhui University. His research interests include deep reinforcement learning, multiagent reinforcement learning, transfer reinforcement learning, and robotic systems.



Wanyi Yao was born in 2002. She received her Bachelor's degree from the School of Mathematics at Hefei University of Technology in 2018 and is currently pursuing a Master's degree in Artificial Intelligence at Anhui University. Her research interests include reinforcement learning, representation learning and causality.



Keke Yang was born in 1998 and pursued undergraduate studies at Anhui Agricultural University from 2017 to 2021, earning a Bachelor of Engineering degree. Continuing his academic journey, he attended the School of Artificial Intelligence at Anhui University from 2022 to 2025, obtaining a Master of Engineering degree. His research interests include reinforcement learning and robotic arm control.



Lu Ren received her Ph.D. degree in control science and engineering from Southeast University in 2021. She is currently an Associate Professor at the School of Artificial Intelligence, Anhui University. Her research interest covers synchronization of complex dynamical networks, deep reinforcement learning, multi-agent reinforcement learning.



Changyin Sun received the bachelor's degree in applied mathematics from the College of Mathematics, Sichuan University, Chengdu, China, in 1996, and the M.S. and Ph.D. degrees in electrical engineering from Southeast University, Nanjing, China, in 2001 and 2004, respectively. He is currently a Professor with the School of Artificial Intelligence, Anhui University, Hefei, China. His current research interests include deep reinforcement learning, intelligent control, and embodied intelligent systems.